腾讯安全 | 云鼎实验室

锋刃无影
御见未来

腾讯安全沙龙第4期（长沙站）

组件安全: AI检测与漏洞利用路径预测

广度更宽精度更深的代码审计工具

关于我
About me

ID: qiqing

from: 顺丰科技蓝军

本次分享您将看到

从困境到解决方案的完整思路演进

第一部分：困境回顾

- 依赖的"冰山"与未知风险
- 在"黑盒依赖"的黑暗地图上航行

第二部分：解决方案

- 思路演进：点亮地图，再画路径
- AI驱动的自动化系统架构
- AI智能验证与路径预测

第三部分：开源例子

- 实战演示

我们的起点：依赖的"冰山"与未知的风险

故事的起点 - 深夜拿到源码后的困惑

真实项目中的依赖复杂度

```
<!-- Maven Dependencies -->
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-web</artifactId>
  </dependency>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
  </dependency>
  <dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
  </dependency>
  <dependency>
```

66
第三方依赖

27
实际使用的依赖

39
未使用的依赖

依赖的"冰山模型"

我们最大的恐惧，并非grep不出结果，而是面对庞大的第三方组件，我们根本不知道风险在哪里。就像手握一份藏宝图，却发现它大部分区域都是黑的。

真正的困境：在"黑盒依赖"的黑暗地图上航行

问题的本质不是效率，而是可见性

核心问题

"所以，问题的本质不是'**单点审计**'效率低。而是我们完全缺乏对依赖项风险的**可见性**。"

"每一个第三方组件，都是我们地图上一块未被点亮的、充满未知危险的区域。"

航海比喻

"我们的审计过程，就像是在一片**黑暗的海域**中航行："

- 不知道哪里有暗礁（漏洞）
- 更不知道宝藏（终点）在何方
- 我们缺的不是船，而是地图和罗盘

我们的现状：几乎全黑的依赖地图

95%+
未知风险区域

5%
可见业务代码

第三方组件的挑战

搜集的Sink可能永远都不够

核心问题

组件数量庞大

- Maven Central 超过400万个组件
- 每天新增数千个版本
- 第三方组件层出不穷
- 我们不可能所有的都认识

Sink搜集困难

- 每个组件都有独特的API
- 危险函数命名不统一
- 文档不完整或缺失
- 搜集的Sink可能永远都不够

有CVE的例子

Nexus 表达式注入

CVE-2022-29078　CVSS 8.8

```
// Nexus Repository Manager
ScriptEngine.eval(userInput)
// 攻击载荷
${java.lang.Runtime.getRuntime().exec("calc")}
```

通过脚本管理功能执行任意代码

甚至还有没有CVE的

Activiti 工作流引擎

无CVE编号　潜在风险

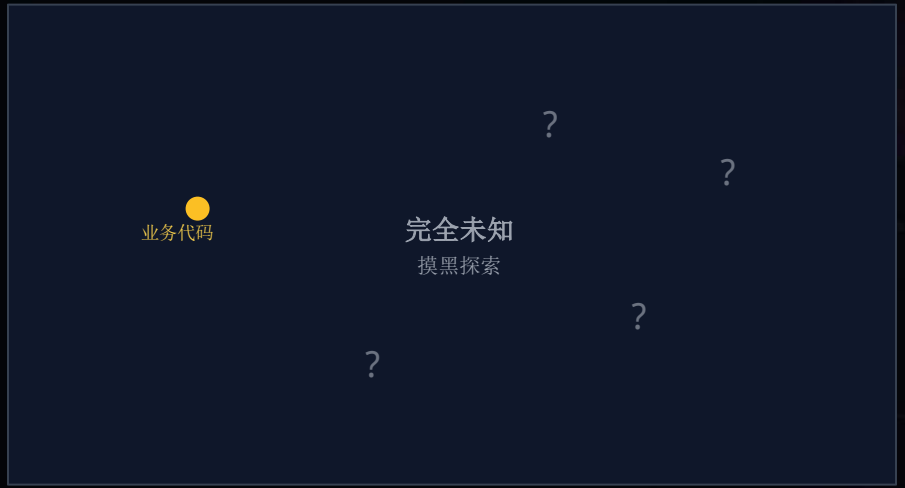
```
// Activiti 表达式执行
ExpressionManager.createExpression()
UelExpressionCondition.evaluate()
// 可能的攻击向量
${T(java.lang.Runtime).getRuntime().exec("cmd")}
```

UEL表达式可能导致代码执行

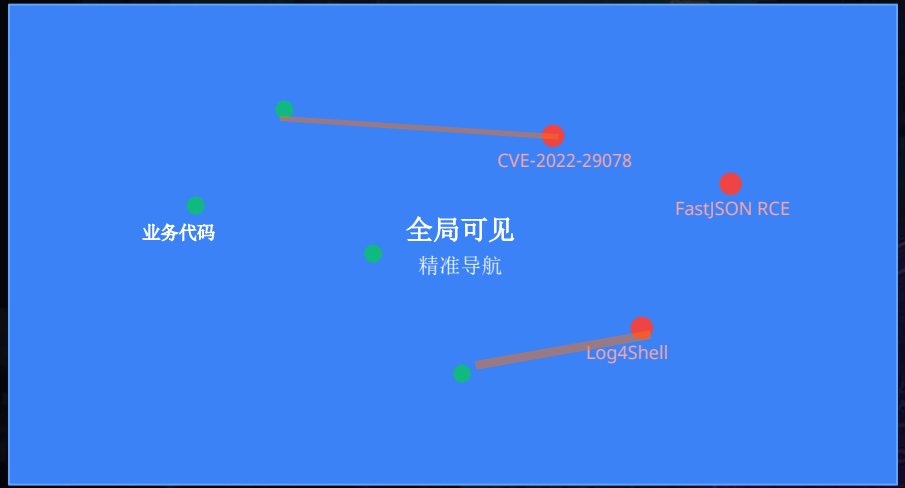
思路的转变：先"点亮地图"，再"预测路径"

从黑暗摸索到精准导航的思维跃迁

之前：黑暗地图



现在：点亮地图



1 第一步：点亮地图

- 自动化情报搜集
- AI智能验证
- 标出所有危险区域

2 第二步：预测路径

- 描绘攻击路线
- 识别关键节点

完整系统架构：四大核心模块

从依赖识别到漏洞扫描的全流程自动化系统

1 获取依赖

开源代码
闭源代码

依赖解析
Maven/Ant/Gradle/本地

2 搜索漏洞

多源聚合
GitHub GHSA/Maven中央仓库

智能验证
Sonatype/无CVE高危依赖

3 AI风险分类

生效分析
引入即生效/函数使用生效

风险评估
配置使用生效/特殊组合生效

4 扫描规则

详细描述
生成风险代码/标记代码漏洞点

规则生成
聚合漏洞点/创建CodeQL规则

识别项目中的依赖与实际使用情况

智能化Maven项目依赖分析与优化

1 POM文件解析

- 解析pom.xml声明的依赖
- 处理Properties变量
- 支持多模块项目

2 Java源码分析

- 扫描import语句
- 识别实际使用的包
- 统计使用频率

3 闭源依赖分析

- 精确匹配→模糊匹配
- 启发式匹配算法
- 转换maven格式

分析统计结果

66 声明的依赖	27 实际使用
39 未使用依赖	282 第三方包

基本统计：
Java文件数量：1101
使用的第三方包：282个
实际使用的依赖：27个
未使用的依赖：39个

依赖使用情况

实际使用的依赖:

- ✓ jackson-databind:2.17.2
- ✓ spring-web:6.1.14
- ✓ freemarker:2.3.33

未使用的依赖:

- ✗ commons-lang3:3.12.0
- ✗ guava:31.1-jre

多维度搜索体系

权威数据源 + 社区发现 + AI增强的核心思路

权威数据源

准确数据保障

- 官方CVE数据库
- 经过验证的高可信度数据源
- 确保漏洞信息的准确性

高准确率

社区发现

发现无CVE漏洞

- 挖掘尚未分配CVE编号的漏洞
- 从开发者讨论中发现安全问题
- 提前识别潜在的安全风险

风险收集

AI增强

更广维度搜索

- 语义理解扩展搜索范围
- 智能关联相似漏洞模式
- 从更广的维度发现安全威胁

广度覆盖

权威验证
官方数据源

深度搜索
社区挖掘

AI智能
语义扩展

多维度搜索体系

权威数据源 + 社区发现 + AI增强的核心思路

GitHub Advisory Database / GitHub Reviewed / CVE-2021-44228

Remote code injection in Log4j

Critical severity | GitHub Reviewed | Published on Dec 10, 2021 to the GitHub Advisory Database • Updated on May 9

Vulnerability details | Dependabot alerts 0

Package	Affected versions	Patched versions
com.guicedee.services:log4j-core (Maven)	<= 1.2.1.2-jre17	None
org.apache.logging.log4j:log4j-core (Maven)	>= 2.13.0, < 2.15.0	2.15.0
	>= 2.4, < 2.12.2	2.12.2
	>= 2.0-beta9, < 2.3.1	2.3.1
org.ops4j.pax.logging:pax-logging-log4j2 (Maven)	>= 1.8.0, < 1.9.2	1.9.2
	>= 1.10.0, < 1.10.8	1.10.8
	>= 1.11.0, < 1.11.10	1.11.10
	>= 2.0.0, < 2.0.11	2.0.11
org.xbib.elasticsearch:log4j (Maven)	= 6.3.2.1	None
uk.co.nichesolutions.logging.log4j:log4j-core (Maven)	= 2.6.3-CUSTOM	None

github.com/Activiti/Activiti/issues/3249

Closed **Activiti have a arbitrary code execution vulnerability** #3249

```
package com.me7ell.demoactiviti7;
import java.util.HashMap;
import java.util.Map;
import org.activiti.engine.*;
import org.activiti.engine.repository.Deployment;
import org.activiti.engine.runtime.ProcessInstance;

public class testBpmn {
    public static void main(String[] args) {
        try{
            ProcessEngine engine = ProcessEngines.getDefaultProcessEngine();
            System.out.println(engine);
            RepositoryService repositoryService = engine.getRepositoryService();
            RuntimeService runtimeService = engine.getRuntimeService();
            Deployment deployment = repositoryService.createDeployment().addClasspathResource("process.bpmn20.
            System.out.println("ID: "+deployment.getId());
            System.out.println("Name: "+deployment.getName());
            Map<String, Object> vars = new HashMap<String, Object>();
            //vars.put("myBean", new String());
            // start
            ProcessInstance pi = runtimeService.startProcessInstanceByKey("process", vars);

        }catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

fastjson 1.2.67 有哪些漏洞

fastjson 1.2.67 vulnerabilities security issues — 10 results

- com.alibaba:fastjson 1.2.67 vulnerabilities snyk.io
- CVE-2022-25845 - Fastjson RCE vulnerability analysis jfrog.com
- Fastjson 1.2.67 Remote Code Execution Vulnerability Scanner s4e.io
- Module "Fastjson 1.2.67 - Remote Code Execution" vidocsecurity.com
- Fastjson 1.2.62 and Earlier Remote Code Execution Vulnerability Threat Alert - ... nsfocusglobal.com
- nuclei-templates/http/vulnerabilities/fastjson/fastjson-1-2-67-rce.yaml at main · pro... github.com
- com.alibaba:fastjson vulnerabilities | Snyk snyk.io

fastjson 1.2.67 CVE 漏洞列表 — 10 results

基于搜索结果，fastjson 1.2.67 存在以下主要安全漏洞：

主要漏洞

1. 远程代码执行漏洞 (CVE-2022-25845) 这是一个高严重性的远程代码执行漏洞，CVSS评分为8.1，允许攻击者绕过AutoTypeCheck机制执行任意代码 JFrog Vidoc Security Lab 。该漏洞的严重程度被归类为关键级别，CVSS评分为10 Module "Fastjson 1.2.67 - R... 。

2. 反序列化远程代码执行 该漏洞涉及通过反序列化恶意构造的JSON数据进行远程代码执行。攻击者可以通过向受影响的应用程序发送特制的JSON请求来利用此漏洞，从而在目标系统上执行任意代码 Fastjson 1.2.67 Remote Cod... 。

AI区分风险类型

Gemini 2.0 Flash驱动的智能风险分析

多维度分析
CVE、严重性、影响版本

来源置信度调整
根据验证结果调整

实时验证
Grounding搜索分析

详细推理
完整分析过程

智能提取
结构化漏洞信息

Gemini 2.0 Flash
最新AI模型

Grounding
实时搜索验证

智能分析
多维度评估

AI区分风险类型

Gemini 2.0 Flash驱动的智能风险分析

community_vulnerabilities / 9

id
github_issue_1464266232
READONLY TEXT

platform
github_issues
READONLY TEXT

url
https://github.com/finos/messageml-utils/issues/346
READONLY TEXT

title
Dependency Dashboard
READONLY TEXT

summary
github/workflows/cve-scanning
READONLY TEXT

description
This issue lists Renovate updates and detected dependencies. Read the [Dependency Dashboard](https://docs.renovatebot.com/key-concepts/dashboard/) docs to learn more.
[View this repository on the Mend.io Web Portal]
READONLY TEXT

package_name
org.freemarker:freemarker

ecosystem
maven

Try Hydejack, a Jekyll theme for hackers, nerds, and academics ↗

```
信息内容:
- 标题: {vulnerability_data.get('title', 'N/A')}
- 描述: {vulnerability_data.get('description', 'N/A')}
- 严重性: {vulnerability_data.get('severity', 'N/A')}
- 包名: {vulnerability_data.get('package_name', 'N/A')}
- 生态系统: {vulnerability_data.get('ecosystem', 'N/A')}
- 受影响版本: {vulnerability_data.get('affected_versions', 'N/A')}
- CWE分类: {vulnerability_data.get('cwe_ids', 'N/A')}

**严格判断标准: **

**必须满足的条件 (全部满足才算真实漏洞): **
1. **是真正的安全漏洞**: 必须是可被恶意利用的安全缺陷
2. **是目标软件包本身的漏洞**: 不是依赖库或第三方组件的漏洞
3. **包含明确的技术细节**: 有具体的技术信息支撑

**明确排除的情况: **
1. **依赖库漏洞**:
   - log4j、commons-fileupload、guava、mybatis、batik等第三方库的漏洞
   - 任何提到"update xxx to fix vulnerabilities"的依赖更新问题
   - 传递依赖的安全问题

2. **非安全漏洞问题**:
   - 配置问题、使用问题、部署问题
   - 功能请求、特性讨论
   - 性能问题、兼容性问题
   - 文档问题、示例问题

3. **模糊或无具体信息**:
   - 仅提及"security issue"但无具体描述
   - 仅询问如何披露漏洞
   - 仅提到CVE编号但无技术细节
   - 一般性安全讨论
```

AI智能构建风险场景

基于真实漏洞代码的智能风险场景构建

实时搜索
获取真实漏洞代码

智能补偿机制
AI自动补偿

三层验证
多重质量保障

标记代码漏洞点
精确标记危险函数调用点

生成AST语法树
解析代码结构生成抽象语法树

风险场景构建
基于漏洞信息构建漏洞代码

真实代码
基于实际漏洞

AI驱动
智能分析处理

AST分析
结构化解析

AI智能构建风险场景

基于真实漏洞代码的智能风险场景构建

存在代码片段或说明

```
dom4j version prior to version 2.1.1 contains a CWE-91: XML Injection vulnerability in Class: Element. Methods: addElement,
addAttribute that can result in an attacker tampering with XML documents through XML injection.
This attack appear to be exploitable via an attacker specifying attributes or elements in the XML document.
This vulnerability appears to have been fixed in 2.1.1 or later.
```

```
        prompt = f"""Generate a risk scenario for this verified vulnerability:

Vulnerability: {optimized['title']}
Type: {', '.join(optimized['technical_summary']['vulnerability_type'])}
"""

        # 如果有代码片段，用它来生成具体场景
        if optimized['evidence']['code_snippets']:
            snippet = optimized['evidence']['code_snippets'][0]
            prompt += f"\nExploit Example:\n```\n{snippet['content']}\n```\n"

        prompt += f"""
Attack Vectors: {', '.join(optimized['technical_summary']['attack_vectors'])}

Generate a realistic attack scenario that shows:
1. How an attacker could exploit this vulnerability
2. What the impact would be
3. Specific steps to reproduce
```

只有漏洞描述

```
dom4j before 2.0.3 and 2.1.x before 2.1.3 allows external DTDs and External Entities by default,
which might enable XXE attacks. However,
there is popular external documentation from OWASP showing how to enable the safe,
non-default behavior in any application that uses dom4j.
```

```
你的任务是根据提供的漏洞描述，生成一个详细的漏洞点推测。这个漏洞描述是关于一个jar包的。
请仔细阅读以下漏洞描述：
<漏洞描述>
{optimized['description']}
</漏洞描述>
在生成漏洞点推测时，请按照以下步骤进行：
1. 仔细研读整个漏洞描述，理解其中的关键信息。
2. 结合jar包的特点和常见漏洞类型，对漏洞可能出现的位置和原因进行分析和推理。
3. 考虑漏洞可能产生的影响和潜在的风险。
4. 形成初步的漏洞点推测。
5. 再次检查，确保推测合理且全面。
首先，在<思考>标签中详细分析你的推理过程和依据。然后，在<推测>标签中输出详细的漏洞点推测，要尽可能丰富、全面。
<思考>
[在此详细说明你的推理过程和依据]
</思考>
<推测>
[在此输出详细的漏洞点推测]
</推测>
```

AI智能构建风险场景

基于真实漏洞代码的智能风险场景构建

构建风险场景

```python
        # 基于漏洞类型提供特定指导
        if any('xxe' in vtype.lower() or 'xml' in vtype.lower() for vtype in vuln_types):
            prompt += """
Focus on XXE/XML vulnerabilities - show vulnerable XML parsing code like:
- DocumentBuilderFactory without security features
- SAXReader with external entities enabled
- Direct XML parsing without validation
"""
        elif any('expression' in vtype.lower() or 'juel' in vtype.lower() for vtype in vuln_types):
            prompt += """
Focus on Expression Language vulnerabilities - show code like:
- Direct evaluation of user input expressions
- JUEL/EL processing without sandboxing
- Reflection access through expressions
"""
```

```python
        prompt += f"""
Attack Vectors: {', '.join(optimized['technical_summary']['attack_vectors'])}

Generate SDL risk scenario showing:
1. Specific vulnerable code that a developer might write
2. Why this code is vulnerable (root cause)
3. Under what conditions it triggers the vulnerability

This is NOT an attack scenario - it's a developer education scenario showing "how NOT to write code".
Focus ONLY on the vulnerable code, no need for secure code examples to save tokens.

Return JSON with these exact fields:
{{
  "vulnerable_code": "具体的Java/XML代码示例，展示漏洞是如何在代码中出现的",
  "vulnerability_trigger": "什么条件会触发这个漏洞",
  "root_cause": "从代码层面分析为什么会有这个漏洞",
  "sdl_analysis": "从SDL角度的分析",
  "code_review_points": ["代码审查时应该注意的点"],
  "can_provide_code": true
}}

If you cannot provide specific vulnerable code examples, set can_provide_code to false.
"""
```

验证代码

```
/jdk1.8/bin/javac -cp "目标/lib或者目标.jar" -d ./verify /generated/code/*.java
```

创建扫描规则

从AST关键节点到CodeQL Sink定义

AST关键节点提取

漏洞函数调用

识别危险函数调用节点

CodeQL Sink定义

Source定义

CodeQL自带

Sink定义

定义危险函数调用点

CodeQL通用模板示例

```
private string targetPackageName() { result= "java.io"}
private string targetClassName() { result= "FileOutputStream"}
private string targetMethodName() { result= "write"}
class CustomSink extends DataFlow::Node{
  CustomSink() {
    exists(MethodCallcall, Methodcallee, RefTypedeclaringType |
callee = call. getMethod() and
callee. hasName(targetMethodName()) and
declaringType = callee. getDeclaringType() and
declaringType. hasQualifiedName(targetPackageName(), targetClassName()) and
      this.asExpr() = call.getAnArgument()
)
}
}
```

实战演示

FreeMarker漏洞分析完整流程展示

Step 1

漏洞收集

```
Step 2: Running comprehensive analysis (Stage 3)...
5-06-12 10:33:14,051 - __main__ - INFO - 数据库表结构初始化完成
5-06-12 10:33:14,052 - __main__ - INFO - 从config.py中读取GitHub token
5-06-12 10:33:14,052 - database.vulnerability_db - INFO - 数据库初始化完成: vulnerabilities.db
5-06-12 10:33:14,052 - services.github_issues_service - INFO - 使用config.py中的GitHub token
5-06-12 10:33:14,052 - services.github_repository_search_service - INFO - 使用config.py中的GitHub token
5-06-12 10:33:14,053 - database.vulnerability_db - INFO - 数据库初始化完成: vulnerabilities.db
5-06-12 10:33:14,053 - __main__ - INFO - 综合漏洞聚合系统已初始化
5-06-12 10:33:14,053 - services.gemini_ai_service - INFO - 代理已设置: http://127.0.0.1:7890
5-06-12 10:33:14,054 - services.gemini_ai_service - INFO - Gemini AI 服务已初始化, 支持 40 个API密钥
5-06-12 10:33:14,054 - __main__ - INFO - Gemini AI服务已初始化, 使用 40 个API密钥
5-06-12 10:33:14,054 - __main__ - INFO - 🔍 开始阶段三: 社区发现 - org.freemarker:freemarker
5-06-12 10:33:14,054 - __main__ - INFO - 📥 搜索 GitHub Issues...
5-06-12 10:33:14,056 - services.search_engine_service - INFO - 使用代理: HTTP=http://127.0.0.1:7890, HTTPS=https://1
0.1:7890
5-06-12 10:33:14,056 - services.search_engine_service - INFO - 通过Google Custom Search API发现 org.freemarker:freem
的GitHub仓库
5-06-12 10:33:18,678 - services.search_engine_service - INFO - Google Custom Search API发现 21 个仓库
5-06-12 10:33:18,678 - services.search_engine_service - INFO - 选择最佳仓库: apache/freemarker
5-06-12 10:33:19,242 - services.github_issues_service - INFO - 通过搜索引擎发现Java仓库: apache/freemarker
5-06-12 10:33:19,717 - services.github_issues_service - INFO - 在仓库 apache/freemarker 中搜索安全问题
5-06-12 10:33:24,591 - services.github_issues_service - INFO - 在仓库 apache/freemarker 中找到 0 个唯一的安全相关iss
5-06-12 10:33:24,592 - __main__ - INFO - 📥 搜索 GitHub Issues...
5-06-12 10:33:31,543 - services.github_issues_service - INFO - 从GitHub Issues找到 24 个潜在安全问题
5-06-12 10:33:31,543 - __main__ - INFO - GitHub Issues 发现 24 个相关问题
5-06-12 10:33:31,543 - __main__ - INFO - 🔍 搜索 GitHub Repositories...
5-06-12 10:33:35,777 - services.github_repository_search_service - INFO - 找到 6 个可能相关的仓库
5-06-12 10:33:35,777 - services.github_repository_search_service - INFO - 搜索仓库 https-github-com-Rama24/peretesan
s全issues
```

Step 2

AI验证

```
🔍 开始AI验证: Snyk reports vulnerability for version 2.4.4...
   ✅ 真实 [ai_optimized] 0.90 - No CVE - high detected in org.freemarker:freemarke...

📝 优化后的提示词统计:
   - 总Token数: 153
   - 原始长度: 329 字符
   - 压缩比: 186.0%
2025-06-12 10:35:00,009 - INFO - 🔷 尝试使用Gemini API...
2025-06-12 10:35:00,054 - INFO - 发送请求到 Gemini (尝试 1/3)
2025-06-12 10:35:00,054 - INFO - AFC is enabled with max remote calls: 10.
2025-06-12 10:35:03,161 - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-fl
ash:generateContent "HTTP/1.1 200 OK"
2025-06-12 10:35:03,163 - INFO - AFC remote call 1 is done.
2025-06-12 10:35:03,163 - INFO - Gemini请求成功, 响应长度: 1010, Grounding来源: 0
   ✅ 真实 [ai_optimized] 0.80 - Claro (SNYK)- High Vulnerabilities...
2025-06-12 10:35:03,626 - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-fl
ash:generateContent "HTTP/1.1 200 OK"
2025-06-12 10:35:03,628 - INFO - AFC remote call 1 is done.
2025-06-12 10:35:03,629 - INFO - Gemini请求成功, 响应长度: 1235, Grounding来源: 4
   🔷 使用 GEMINI API
🎯 在低SDL风险场景中...
2025-06-12 10:35:03,650 - INFO - 🔷 尝试使用Gemini API...
2025-06-12 10:35:03,688 - INFO - 发送请求到 Gemini (尝试 1/3)
2025-06-12 10:35:03,688 - INFO - AFC is enabled with max remote calls: 10.
2025-06-12 10:35:04,161 - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-fl
ash:generateContent "HTTP/1.1 200 OK"
2025-06-12 10:35:04,163 - INFO - AFC remote call 1 is done.
2025-06-12 10:35:04,163 - INFO - Gemini请求成功, 响应长度: 1207, Grounding来源: 4
   🔷 使用 GEMINI API
🎯 在低SDL风险场景中...
2025-06-12 10:35:04,164 - INFO - 🔷 尝试使用Gemini API...
2025-06-12 10:35:04,205 - INFO - 发送请求到 Gemini (尝试 1/3)
```

Step 3

组装扫描规则

```
CustomSink() {
  exists(MethodCall call, Method callee, RefType declaringType |
    callee = call.getMethod() and
    callee.hasName(targetMethodName()) and
    declaringType = callee.getDeclaringType() and
    declaringType.hasQualifiedName(targetPackageName(), targetClassName()) and
    this.asExpr() = call.getAnArgument()
  )
}
}

module DebugConfig implements DataFlow::ConfigSig {
  Quick Evaluation: isSource
  predicate isSource(DataFlow::Node source) {
    source instanceof RemoteFlowSource
  }
  Quick Evaluation: isSink
  predicate isSink(DataFlow::Node sink) {
    sink instanceof CustomSink
  }
```

Step 4

依赖扫描

```
✅ 实际使用的依赖 (27个):
  ✓ cn.smallbun.screw:screw-core:1.0.5
  ✓ com.alipay.sdk:alipay-easysdk:2.2.3
  ✓ com.blueconic:browscap-java:1.4.4
  ✓ com.fasterxml.jackson.core:jackson-databind:2.17.2
  ✓ com.github.therapi:therapi-runtime-javadoc-scribe:0.15.0
  ✓ com.mysql:mysql-connector-j:9.0.0
  ✓ com.zaxxer:HikariCP:5.1.0
  ✓ commons-codec:commons-codec:1.17.1
  ✓ fr.opensagres.xdocreport:fr.opensagres.poi.xwpf.converter.xhtml:2.1.0
  ✓ jakarta.servlet:jakarta.servlet-api:6.1.0
  ✓ org.apache.commons:commons-collections4:4.4
  ✓ org.apache.logging.log4j:log4j-api:2.24.0
  ✓ org.apache.lucene:lucene-analysis-smartcn:9.11.1
  ✓ org.apache.poi:poi-ooxml:5.3.0
  ✓ org.freemarker:freemarker:2.3.30
  ✓ org.hibernate.hibernate.search:hibernate-search-mapper-orm:7.2.1.Final
  ✓ org.hibernate.tool:hibernate-tools-orm:6.6.0.Final
  ✓ org.jsoup:jsoup:1.18.1
  ✓ org.junit.jupiter:junit-jupiter-engine:5.11.0
  ✓ org.lionsoul:ip2region:2.7.0
```

Step 5

漏洞Sink

```
},
"ai_analysis": {
  "vulnerability_sink": "template.process(data, writer)",
  "vulnerable_functions": [
    "import freemarker.template.Configuration;",
    "import freemarker.template.Template;",
    "import freemarker.template.TemplateException;",
    "import java.io.IOException;",
    "import java.io.StringWriter;",
    "import java.util.Map;",
    "Configuration configuration = new Configuration(Configuration.VERSION_2_3_31); // Example
    Configuration",
    "Template template = configuration.getTemplate(templateName + \".ftl\");",
    "StringWriter writer = new StringWriter();",
    "template.process(data, writer);"
  ],
  "vulnerability_points": [
    "data参数中的值可能被用户控制，导致可以加载任意模板。",
    "data参数中的值可能被用户控制，如果data中的值包含对java.security.ProtectionDomain.getClassLoader的调用，则可能
    导致代码执行。",
    "Freemarker模板引擎允许在模板中使用表达式，如果用户可以控制模板内容或传递给模板的数据，则可能执行任意代码。"
  ],
```

Step 6

风险场景

```
   ID: github_issue_1116032686
   标题: Cx3349B186-093f @ Maven-org.freemarker:freemarker-...
   包名: org.freemarker:freemarker
   严重程度: 未知
\n🔍 验证信息:
   验证结果: ✅ 真实漏洞
   置信度: 0.90
   生成来源: GROUNDING
   质量评分: 1.0
\n📝 有漏洞的代码示例:
```\n<#assign ex = "freemarker.template.utility.Execute"?new()>
${ ex("id") }\n```

\n🎯 SDL场景 #3
──
📋 漏洞信息:
   ID: 1
   标题: Server-Side Template Injection (SSTI)
   包名: org.freemarker
   严重程度: high
\n🔍 验证信息:
   验证结果: ✅ 真实漏洞
   置信度: 0.90
   生成来源: GROUNDING
\n📝 有漏洞的代码示例:
```\npublic String generateReport(String templateName, Map<String, Object> data) throws IOException, TemplateException {
  Template template = configuration.getTemplate(templateName + ".ftl");
  StringWriter\n... (完整代码请使用导出功能)\n```

\n🎯 SDL场景 #4
```

多维搜索
收集漏洞信息

AI验证
智能分析验证

规则生成
自动化扫描

风险评估
场景构建

谢谢